

# CI/CD Pipeline - Horizon Medical WSS

## Descripción General

El proyecto **hrzmed\_wss** implementa un pipeline de Integración Continua / Entrega Continua (CI/CD) usando **GitHub Actions**. Este pipeline asegura la calidad del código, ejecuta tests automáticamente y construye imágenes Docker listas para despliegue.

## Arquitectura del Pipeline

Push/PR → Lint → Unit Tests (Node 18/20/22) → Integration Tests (MongoDB) → Coverage → Docker Build

### Workflows

| Workflow             | Archivo                                   | Trigger                                                                     | Descripción                                       |
|----------------------|-------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------|
| CI - Tests & Linting | <code>.github/workflows/ci.yml</code>     | Push/PR a <code>main</code> , <code>develop</code> , <code>feature/*</code> | Linting, tests unitarios, integración y cobertura |
| Docker Build & Push  | <code>.github/workflows/docker.yml</code> | Push a <code>main</code> , <code>develop</code> , tags <code>v*.*.*</code>  | Build Docker, health check, push a GHCR           |

## Workflow 1: CI - Tests & Linting

### Jobs

#### Lint (ESLint)

- Ejecuta ESLint con la configuración del proyecto ( `.eslintrc.json` )
- Genera un reporte JSON en `reports/eslint-report.json`
- El reporte se sube como artifact de GitHub Actions

#### Unit Tests

- Se ejecutan en una **matrix** de Node.js: `18` , `20` , `22`
- Garantiza compatibilidad multi-versión
- Incluye tests de: autenticación, validación ECG, compresión de señales, persistencia

#### Integration Tests

- Levanta un servicio MongoDB 7 como **service container**
- Ejecuta tests end-to-end con conexiones WebSocket reales
- Valida flujos completos: conexión → envío de datos → persistencia → consulta API

## Coverage Report

- Genera reporte de cobertura con **c8**
- Formatos: texto (consola), LCOV (para herramientas de cobertura), JSON summary
- Se sube como artifact para descarga

## Workflow 2: Docker Build & Push

### Jobs

#### Docker Build

1. Configura Docker Buildx para builds optimizados
2. Login a GitHub Container Registry (GHCR)
3. Extrae metadata para tags automáticos (branch, PR, semver, SHA)
4. Construye la imagen Docker
5. Ejecuta un **health check** del contenedor:
  - Inicia el contenedor en modo test
  - Verifica el endpoint `/health` responda HTTP 200
  - Falla el pipeline si el health check no pasa
6. Push de la imagen a GHCR (solo en push, no en PRs)

#### Docker Compose Validation

- Valida la sintaxis de `docker-compose.yml`
- Ejecuta **Hadolint** para lint del Dockerfile

### Tags de Imágenes Docker

| Evento                      | Tags generados                                                   |
|-----------------------------|------------------------------------------------------------------|
| Push a <code>main</code>    | <code>main</code> , <code>sha-abc1234</code>                     |
| Push a <code>develop</code> | <code>develop</code> , <code>sha-abc1234</code>                  |
| Tag <code>v1.2.3</code>     | <code>1.2.3</code> , <code>1.2</code> , <code>sha-abc1234</code> |
| Pull Request                | <code>pr-42</code>                                               |

## Linting (ESLint)

### Configuración

El archivo `.eslintrc.json` define las reglas de calidad del código:

- **Base:** `eslint:recommended`
- **Entorno:** Node.js, ES2022
- **Reglas principales:**
  - `no-eval` , `no-implicit-eval` , `no-new-func` : Seguridad contra inyección de código
  - `eqeqeq` : Comparaciones estrictas

- `prefer-const` , `no-var` : Mejores prácticas ES6+
- `semi` , `quotes` , `indent` : Consistencia de estilo
- Tests tienen reglas más relajadas (archivos en `tests/` )

## Comandos

```
# Ejecutar linting
npm run lint

# Ejecutar linting y auto-corregir
npm run lint:fix

# Generar reporte JSON
npm run lint:report
```

## Cobertura de Tests

### Herramienta: c8

Se usa **c8** (built-in V8 coverage) para generar reportes de cobertura sin necesidad de instrumentación.

## Comandos

```
# Todos los tests con cobertura
npm run test:coverage

# Solo tests unitarios con cobertura
npm run test:coverage:unit
```

## Reportes Generados

| Formato      | Ubicación                                    | Uso                                               |
|--------------|----------------------------------------------|---------------------------------------------------|
| Texto        | Consola (stdout)                             | Revisión rápida                                   |
| LCOV         | <code>coverage/lcov.info</code>              | Integración con herramientas (Codecov, SonarQube) |
| JSON Summary | <code>coverage/coverage-summary.json</code>  | Badges dinámicos, CI checks                       |
| HTML         | <code>coverage/lcov-report/index.html</code> | Navegación visual detallada                       |

## Cobertura Actual

| Métrica    | Porcentaje |
|------------|------------|
| Statements | 90.86%     |
| Branches   | 75.91%     |
| Functions  | 89.65%     |
| Lines      | 90.86%     |

---

## Ejecución Local

### Pre-requisitos

- Node.js  $\geq 18$
- npm  $\geq 8$
- Docker y Docker Compose (para tests de integración con MongoDB real)

### Comandos Rápidos

```
# Instalar dependencias
npm install

# Ejecutar linting
npm run lint

# Ejecutar todos los tests
npm test

# Ejecutar tests con cobertura
npm run test:coverage

# Pipeline CI completo (lint + tests)
npm run ci

# Pre-commit check (lint + unit tests)
npm run precommit
```

## Variables de Entorno en CI

| Variable      | Valor en CI        | Descripción                |
|---------------|--------------------|----------------------------|
| NODE_ENV      | test               | Modo de ejecución          |
| JWT_SECRET    | ci-test-secret-... | Secret para JWT en tests   |
| JWT_ALGORITHM | HS256              | Algoritmo de firma         |
| AUTH_ENABLED  | false              | Desactiva auth para tests  |
| LOG_LEVEL     | error              | Solo errores en CI         |
| MONGO_URI     | mongodb://...      | URI de MongoDB de servicio |
| DBNAME        | hrzmed_test        | Base de datos de pruebas   |

## Secrets de GitHub Requeridos

| Secret       | Descripción                          | Requerido para                 |
|--------------|--------------------------------------|--------------------------------|
| GITHUB_TOKEN | Automático, proporcionado por GitHub | Push de imágenes Docker a GHCR |

No se requieren secrets adicionales para el pipeline básico. Para integraciones futuras (Codecov, Slack notifications), agregar los secrets correspondientes.

## Badges del Proyecto

Los badges en el README reflejan el estado actual del pipeline:

| Badge                                                                             | Descripción               |
|-----------------------------------------------------------------------------------|---------------------------|
|  | Estado del workflow de CI |
|  | Estado del build Docker   |
|  | Versión mínima de Node.js |
|  | Licencia del proyecto     |
|  | Cobertura de tests        |

## Mejoras Futuras

1. **Codecov Integration:** Subir reportes LCOV a Codecov para badges dinámicos de cobertura
2. **Dependabot:** Configurar actualizaciones automáticas de dependencias
3. **Slack/Teams Notifications:** Notificar al equipo sobre fallos en el pipeline
4. **CD (Continuous Deployment):** Deploy automático a staging tras merge a `develop`
5. **Security Scanning:** Integrar `npm audit` y escaneo de vulnerabilidades en contenedores
6. **Performance Tests:** Añadir benchmarks de latencia WebSocket al pipeline
7. **Branch Protection Rules:** Requerir checks de CI exitosos antes de merge